# Tutorato AsD ~ L2

05/03/2026

**Esercizio 1.** Modificare la subroutine MERGE di MERGE-SORT in modo da non utilizzare le sentinelle.

```
MERGE(A, p, q, r)
 1  n_L = q − p + 1          // length of A[p : q]
 2  n_R = r − q              // length of A[q + 1 : r]
 3  let L[0 : n_L − 1] and R[0 : n_R − 1] be new arrays
 4  for i = 0 to n_L − 1     // copy A[p : q] into L[0 : n_L − 1]
 5      L[i] = A[p + i]
 6  for j = 0 to n_R − 1     // copy A[q + 1 : r] into R[0 : n_R − 1]
 7      R[j] = A[q + j + 1]
 8  i = 0                    // i indexes the smallest remaining element in L
 9  j = 0                    // j indexes the smallest remaining element in R
10  k = p                    // k indexes the location in A to fill
11  // As long as each of the arrays L and R contains an unmerged element,
    //      copy the smallest unmerged element back into A[p : r].
12  while i < n_L and j < n_R
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
18      k = k + 1
19  // Having gone through one of L and R entirely, copy the
    //      remainder of the other to the end of A[p : r].
20  while i < n_L
21      A[k] = L[i]
22      i = i + 1
23      k = k + 1
24  while j < n_R
25      A[k] = R[j]
26      j = j + 1
27      k = k + 1
```

```
MERGE(A, p, q, r)
 1  n_1 = q − p + 1
 2  n_2 = r − q
 3  let L[1 . . n_1 + 1] and R[1 . . n_2 + 1] be new arrays
 4  for i = 1 to n_1
 5      L[i] = A[p + i − 1]
 6  for j = 1 to n_2
 7      R[j] = A[q + j]
 8  L[n_1 + 1] = ∞
 9  R[n_2 + 1] = ∞
10  i = 1
11  j = 1
12  for k = p to r
13      if L[i] ≤ R[j]
14          A[k] = L[i]
15          i = i + 1
16      else A[k] = R[j]
17          j = j + 1
```

```
MERGE-SORT(A, p, r)
1  if p ≥ r                          // zero or one element?
2      return
3  q = ⌊(p + r)/2⌋                   // midpoint of A[p : r]
4  MERGE-SORT(A, p, q)               // recursively sort A[p : q]
5  MERGE-SORT(A, q + 1, r)           // recursively sort A[q + 1 : r]
6  // Merge A[p : q] and A[q + 1 : r] into A[p : r].
7  MERGE(A, p, q, r)
```

**Esercizio 2.** Scrivere uno pseudocodice per l'algoritmo di ricerca binaria visto a lezione e dimostrarne la correttezza.

```
BINARY-SEARCH (A, p, r, x):
  if p ≤ r:
    q = ⌊(p+r)/2⌋
    if A[q] = x:
      return q
    else if A[q] > x:
      return BINARY-SEARCH (A, p, q-1, x)
    else:  else if A[q] < x:
      return BINARY-SEARCH (A, q+1, r, x)
  else:
    return -1
```

- $r = p + 2k$
- $r = p + 2k + 1$

$q = p + k$

$p \to q-1$ $q$ $q+1 \to r$

# CORRETTEZZA

Per induzione su dim $n = r - p + 1$ di A.

★ Se $n = 1 \Rightarrow p = r \ (= q)$ ✓

↳ Se $A[q] > x \rightarrow$ Se $x \in A \Rightarrow x \in A[p : q-1]$

↳ $q = \left\lfloor \frac{p+r}{2} \right\rfloor \overset{p<r}{\leq} \left\lfloor \frac{r+r}{2} \right\rfloor = r \quad \Rightarrow q - 1 = r - 1 < r$ ($q \leq r$, $q-1 < r$)

↳ $n' = (q-1) - p + 1 < r - p + 1 = n$

↳ Se $A[q] < x$ è analogo  Hint: $\left\lfloor \frac{p+r}{2} \right\rfloor \geq \left\lfloor \frac{p+p}{2} \right\rfloor$

**Esercizio 3.** Scrivere uno pseudocodice per l'algoritmo di elevamento a potenza visto a lezione e dimostrarne la correttezza.

```
POWERING(a, n):

    if n=0:
        return 1

    if n è PARI:

        K = POWERING(a, n/2)
        return K·K

    else:   [if n è DISPARI]

        K = POWERING(a, (n-1)/2)
        return a·K·K
```

1. Quando viene usata questa guardia??
   ↳ $\forall n \in \mathbb{N} \setminus \{0\}$

$n\%2=0$ ó $n=2K$

2. Funziona per n<0?
   ↳ NO, ENTRA IN LOOP

## CORRETTEZZA

★ Se $n=0$, $a^n = a^0 = 1$ ✓

↳ Se n è PARI ($n=2m$)

$$a^n = K \cdot K = \text{POW.}\left(a, \overset{m}{\overset{\|}{\frac{n}{2}}}\right) \cdot \text{POW.}\left(a, \overset{m}{\overset{\|}{\frac{n}{2}}}\right) =$$

$$= a^{n/2} \cdot a^{n/2} = a^n$$

$$= a^m \cdot a^m = a^{2m} = a^n$$

↳ Se n è DISPARI ($n=2m+1$)

$$a^n = a \cdot K \cdot K = a \cdot \text{Pow.}\left(a, \overset{m}{\frac{n-1}{2}}\right) \cdot \text{POW.}\left(a, \overset{m}{\frac{n-1}{2}}\right) =$$

$$= a^1 \cdot a^{\frac{n-1}{2}} \cdot a^{\frac{n-1}{2}} = \ldots = a^{2m+1}$$

**Esercizio 4.** Sia $T(n)$ il numero di righe stampate dal seguente algoritmo ricorsivo. Dare un limite asintotico stretto per $T(n)$.

```
function f(n)

if n > 1:
    print(''still going'')      1
    f(n/2)                      T(n/2)
    f(n/2)                      T(n/2)
```

$$T(n) = \begin{cases} \emptyset & , n = 1 \\ 2T(n/2) + 1 & , n > 1 \end{cases}$$

## MASTER THM

$\exists\, n_0 \in \mathbb{N}$ t.c. $f(n) > 0 \ \forall n \geq n_0$

$T(n) = aT(n/b) + f(n)$ , $a \geq 1$, $b > 1$, $f(n)$ asint. pos.

1. Se $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$ cost.,

   allora $T(n) = \Theta(n^{\log_b a})$

2. Se $f(n) = \Theta(n^{\log_b a})$,

   allora $T(n) = \Theta(n^{\log_b a} \log_2 n)$

3. Se $f(n) = \Omega(n^{\log_b a + \varepsilon})$, $\varepsilon > 0$ cost.

   $\wedge$

   $a f(\frac{n}{b}) \leq c f(n)$ per qualche $c < 1$ e "suff. grande"

   allora $T(n) = \Theta(f(n))$

## MASTER THEOREM

$$T(n) = \begin{cases} \emptyset & , n = 1 \\ 2T(n/2) + 1 & , n > 1 \end{cases}$$

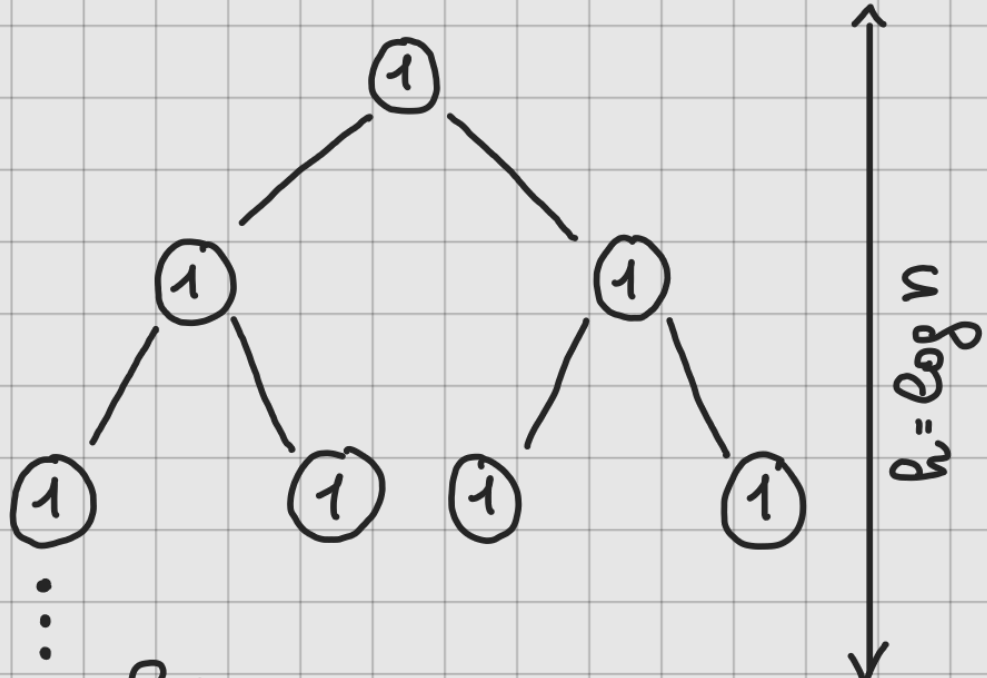
↳ $a = 2 \qquad b = 2 \qquad f(n) = 1$

↳ $n^{\log_b a} = n^{\log_2 2} = 1$

Prendo $\varepsilon = \frac{1}{2}$: $\quad 1 = O\left(n^{1-\frac{1}{2}}\right)$ CASO 1

↳ $T(n) = \Theta(n)$

## ALBERO DI RICORSIONE




$1$
$2$
$2^2$
$\vdots$
$2^{\log n - 1}$
$0$ $2^{\log n} \cdot \emptyset$ foglie

$$\sum_{i=0}^{\log n - 1} 2^i = \ldots = 2^{\log n} - 1$$

SOSTITUZIONE

O_CLAIM  HP: $T(n) \leq cn \ \forall n \geq n_0$

↳ CB: $n=1 \quad T(1)=0 \leq c$ ✓

↳ PI: Sia $n>1$

$$T(n) = 2T\left(\frac{n}{2}\right)+1 \overset{HP}{\leq} 2c\frac{n}{2}+1 = cn+1 \overset{?}{\not\leq} cn$$

⇝ Devo rafforzare HP

O_CLAIM' HP: $T(n) \leq cn - c' \ \forall n \geq n_0$

sottraggo termine di ordine inferiore

CB: $n=1 \quad T(1)=0 \overset{?}{\leq} c-c' \longrightarrow \boxed{c' \leq c}$

PI: Sia $n>1$

$$T(n) = 2T(n/2)+1 \overset{HP}{\leq} 2\left(c\frac{n}{2}-c'\right)+1 =$$

$$= cn-2c'+1 \overset{?}{\leq} cn-c' \longrightarrow \boxed{c' \geq 1}$$

⇝ $n_0 = c = c' = 1$

$\Omega$_CLAIM  HP: $T(n) \geq cn \ \forall n \geq n_0$ $\overset{?}{\not\geq}$

CB: $n=1 \quad T(1)=0 \geq c$

$n=2 \quad T(2)=2T(1)+1=1 \geq c \longrightarrow c \leq 1$

PI: Sia $n \geq 2$

$$T(n) = 2T(n/2) + 1 \overset{HP}{\geq} 2c\frac{n}{2} + 1$$

$$= cn + 1 \overset{?}{\geq} cn \longrightarrow \forall c \in \mathbb{R}^+$$

$\rightsquigarrow n_0 = 2$, $c \in (0;1]$ (e.g. $c = \frac{1}{2}$)

$O_CLAIM + \Omega_CLAIM \Rightarrow T(n) = \Theta(n)$

**Esercizio 5.** Usando il metodo di sostituzione, dimostrare che la soluzione di $T(n) = T(n-1) + n$ è $O(n^2)$ (si assuma che $T(1) = \cancel{0}\ K$

$$T(n) = \begin{cases} K & , n = 1 \\ T(n-1) + n & , n > 1 \end{cases} \qquad T(n) \overset{?}{=} O(n^2)$$

HP: $T(n) \leq cn^2 \quad \forall n \geq n_0$

CB: $n = 1 \quad T(n) = K \leq c \longrightarrow c \geq K$

PI: $T(n) = T(n-1) + n \overset{HP}{\leq} c(n-1)^2 + n =$

$$= cn^2 - 2cn + c + n \overset{?}{\leq} cn^2$$

↳ $2cn - n \geq c \Rightarrow n(2c-1) \geq c$

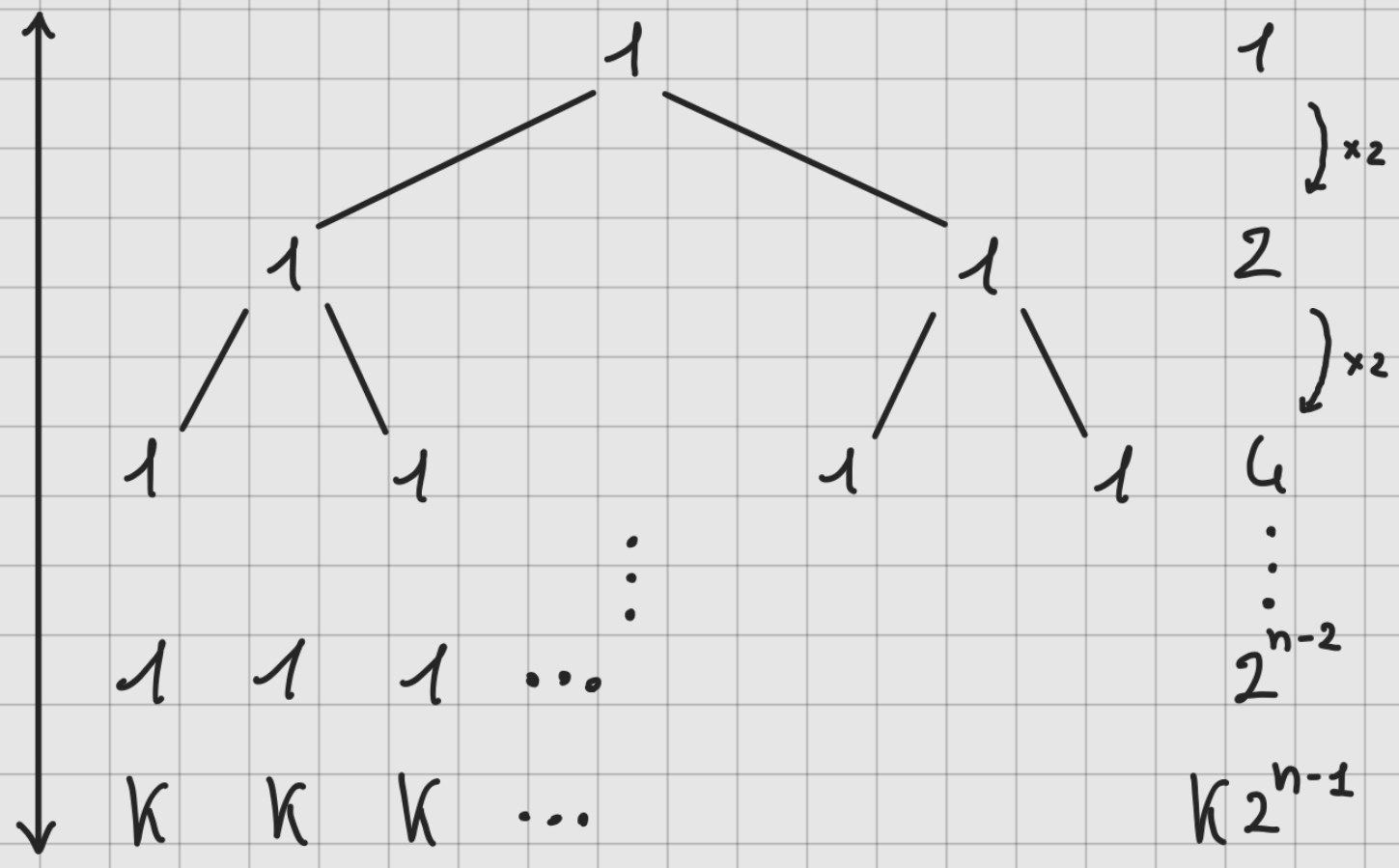
↳ Sicuramente vero per $2c - 1 \geq c \longrightarrow c \geq 1$

⇝ $n_0 = 1$ , $c = \max\{1, K\}$

**Esercizio 6.** Utilizzare un albero di ricorsione per determinare un buon limite asintotico superiore per $T(n)$ e applicare il metodo di sostituzione per verificarlo (si assuma in tutti i casi che $T(1) = ~~c~~ k$):

1. $T(n) = 2T(n-1) + 1$;
2. $T(n) = 2T(n/2) + n \log n$.

1. $T(n) = 2T(n-1)+1 \qquad [T(1)=c]$


1
1
1

Raffozzo HP: $T(n) \leq c2^n - n$

PI: Sia $n > n_0$

$$T(n) = 2T(n-1) + 1 \overset{HP}{\leq} 2\left(c2^{n-1} - (n-1)\right) + 1 =$$

$$= c2^n - 2n + 3 \overset{?}{\leq} c2^n - n$$

↳ $-2n + 3 \leq -n \Rightarrow n \geq 3 \longrightarrow n_0 = 3$

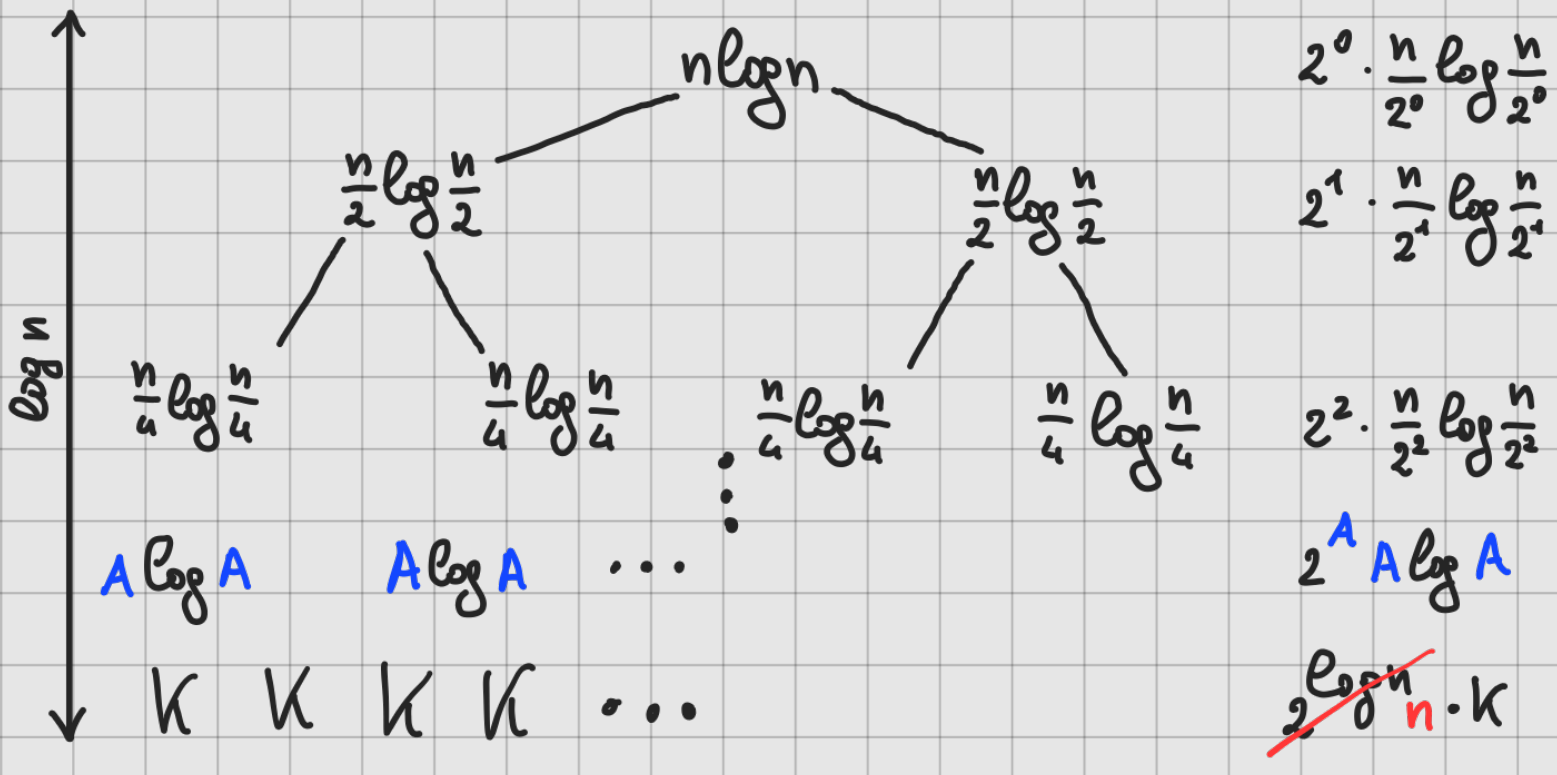
CB: $n = 3 \quad T(3) = 2T(2) + 1 = 2(2\overbrace{T(1)}^{K} + 1) + 1 =$

$$= 4K + 3 \leq c2^3 - 3 = 8c - 3 \longrightarrow c \geq \frac{4K+6}{8}$$

⇝ $n_0 = 3$, $c = \frac{2K+3}{4}$

---

2. $T(n) = 2T(n/2) + n\log n \qquad A = (\log n) - 1$



$$\sum_{i=0}^{\log(n)-1} 2^i \cdot \frac{n}{2^i} \cdot \log \frac{n}{2^i} = \sum_{i=0}^{\log(n)-1} n\log n - n\log 2^i =$$

$$= n\log n(\log n - 1) - n\log 2 \sum_{i=0}^{\log n - 1} i =$$

$$= n\log n \log n - n\log n - n\frac{(\log n - 1)(\log n - 2)}{2} =$$

$$= \ldots = O(n\log n \log n)$$

$$T(n) = O(n\log n \log n)$$

HP: $T(n) \leq cn\log n \log n$

CB: $n = 1 \quad T(1) = K \overset{?}{\leq} c \cdot 1 \cdot 0 \cdot 0$

$n = 2 \quad T(2) = 2K + 2 \overset{?}{\leq} c \cdot 2 \cdot 1 \cdot 1 \longrightarrow c \geq K + 1$

PI: Sia $n > 2$

$$T(n) = 2T(n/2) + n\log n \overset{HP}{\leq} 2c\frac{n}{2}\log\frac{n}{2}\log\frac{n}{2} + n\log n =$$

$$(\log n - 1)^2 = \log^2 n - 2\log n + 1$$

$$= cn\log^2 n - 2cn\log n + cn + n\log n =$$

$$= cn\log^2 n + n(c + \log n - 2c\log n) \overset{?}{\leq} cn\log^2 n$$

$$\hookrightarrow c + \log n - 2c\log n \leq 0 \longrightarrow c \geq 1 \geq \frac{\log n}{2\log n - 1}$$

$\forall n \geq 2$

$\rightsquigarrow n_0 = 2 \quad , \quad c = \max\{k+1, 1\}$

**Esercizio 7.** Consideriamo il problema di calcolare la somma degli elementi di un array di $n$ numeri. Usando il Divide et Impera, fornire un algoritmo che risolva il problema e stimarne la complessità. È migliore dell'algoritmo brute force che scorre l'array?

```
SUM(A, p, r):
    if p = r:
        return A[p]
    else if p < r:
        q = ⌊(p+r)/2⌋
        return SUM(A, p, q) + SUM(A, q+1, r)
```

$T(n) = k\,T\left(\frac{n}{k}\right) + \Theta(1)$ , $k$ fissato

MASTER THM - CASO 1: $T(n) = \Theta(n)$

**Esercizio 8.** Ci è dato un vettore di $n$ numeri e notiamo che alcuni di essi si ripetono. Come possiamo rimuovere tutti i doppioni in $O(n \log n)$ passi?

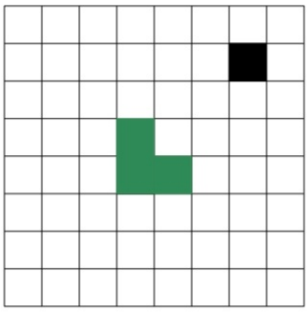
1. Ordino con MERGE-SORT in $\Theta(n \log n)$
2. REMOVE in $\Theta(n)$

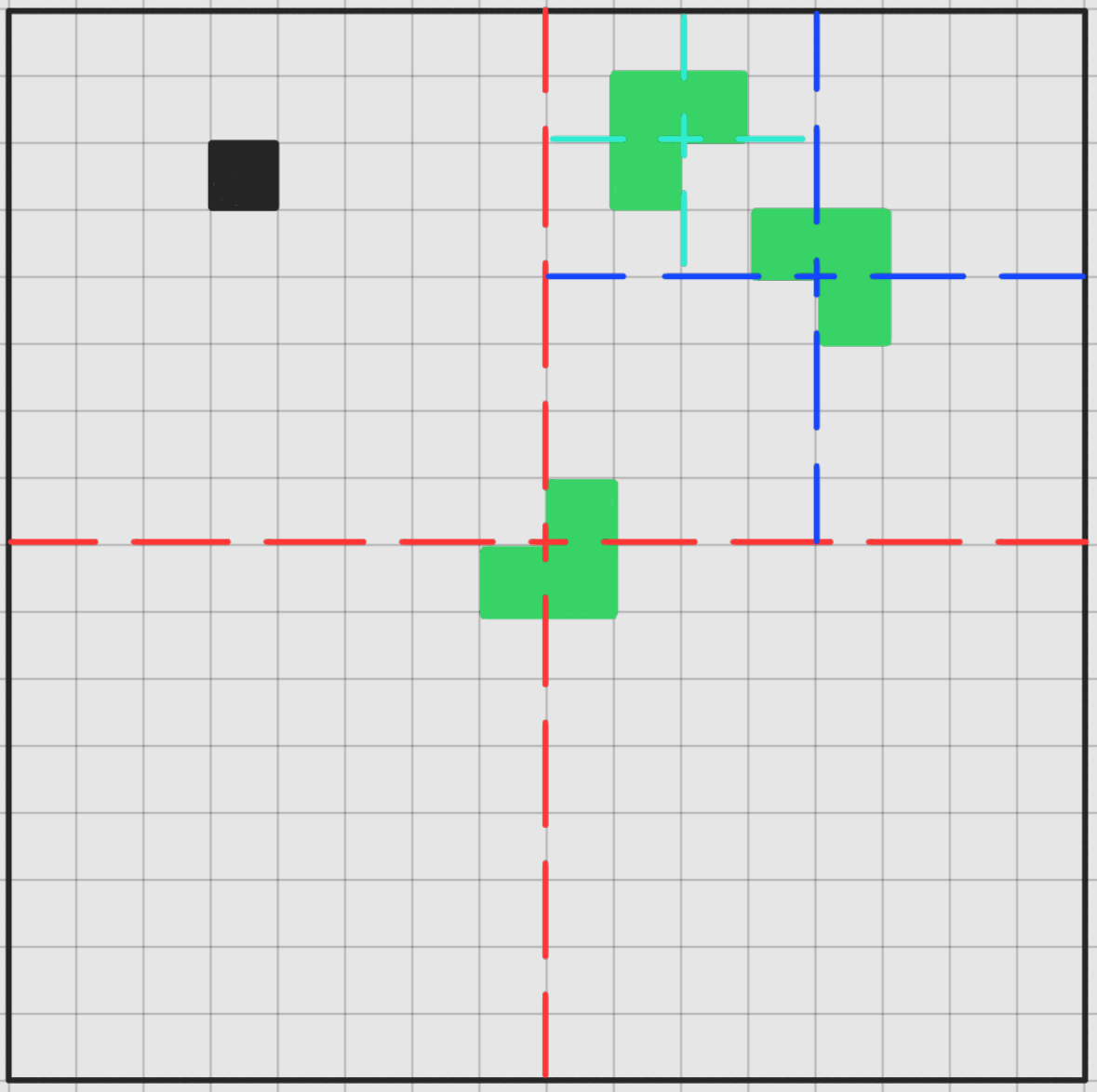
↳ $T(n) = \Theta(n \log n) + \Theta(n) = \Theta(n \log n)$

**Esercizio 9.** Sia $A$ un vettore di $n$ interi distinti e ordinati. Vogliamo determinare se esiste un indice $i$ tale che $A[i] = i$. Fornire un algoritmo con tempo di esecuzione $O(\log n)$.

```
BINARY-SEARCH (A,p,r,x):
  if p≤r:
    q=⌊(p+r)/2⌋
    if A[q]=x:
      return q
    else if A[q]>x:
      return BINARY-SEARCH(A,p,q-1,x)
    else:
      return BINARY-SEARCH(A,q+1,r,x)
  else:
    return -1
```

**Esercizio 10.** Si vuole tassellare una scacchiera $n \times n$ (con $n$ potenza di 2), priva di una casella (per esempio, la casella nera in figura), con dei tasselli a forma di L occupanti tre caselle (tassello verde in figura e sue rotazioni). Fornire un algoritmo con tempo di esecuzione $O(n^2)$.





$$T(n) = 4\,T(n/2) + \Theta(1)$$

MASTER THEOREM → $T(n) = \Theta(n^2)$

C1



L'IDEA É CHE AD OGNI "ITERAZIONE" VOGLIO MANTENERE INVARIATA UNA QUALCHE PROPRIETÁ ⟶ NEL NOSTRO CASO:

LA CONDIZIONE INIZIALE, cioé:

- QUADRATO $2^k \cdot 2^k$ $(k \in \mathbb{N})$
- ESATTAMENTE 1 CELLA GIÁ OCCUPATA

↳ DA QUI POSSIAMO RISOLVERE FACILMENTE

IL CASO BASE:

